MEMORIA DEL PROYECTO DE DISEÑO

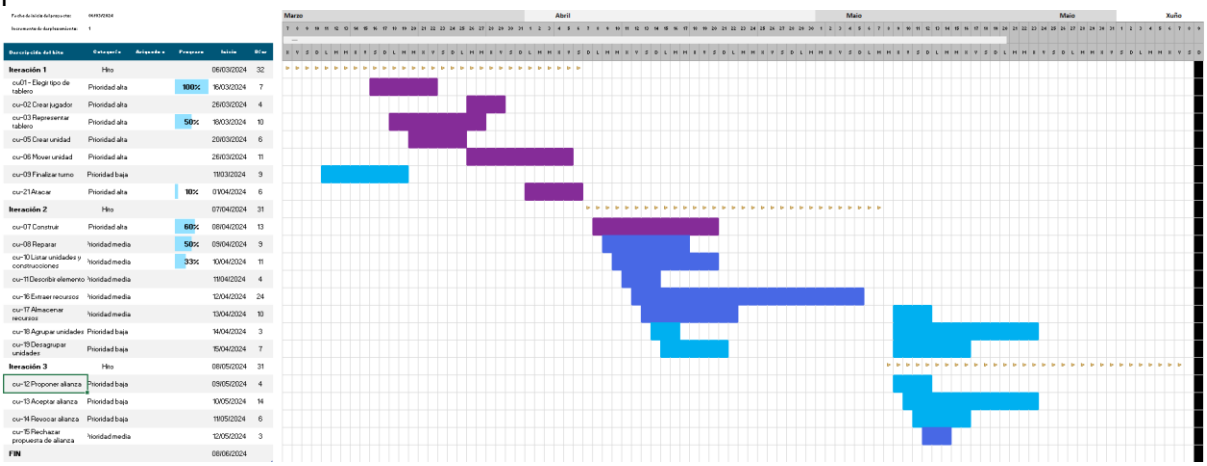
Miguel Carreño Vila
Marcelo Fort Muñoz
Yago García Álvarez
Iván García González

1. Fase de inicio

Con este proyecto se busca crear el diseño necesario para desarrollar un videojuego de estrategia por turnos similar a *Mars Madness* dirigido a partidas de entre 2 y 4 personas. Para ello, a lo largo de las diferentes fases se desarrollarán varios diagramas contenidos en un proyecto de *StarUML*. Esta Fase se superó en la primera sesión de prácticas.

1.1 Planificación:

Para el desarrollo del proyecto, se ha llevado una planificación de los tiempos de diseño de los distintos casos de uso. A continuación mostramos un diagrama de Gantt con la planificación:



Descripción del hito	Categoría	Asignado a	Progreso	Inicio	Días
Iteración 1	Hito			06/03/2024	32
cu01 - Elegir tipo de tablero	Prioridad alta		100%	16/03/2024	7
cu-02 Crear jugador	Prioridad alta			26/03/2024	4
cu-03 Representar tablero	Prioridad alta		50%	19/03/2024	10
cu-05 Crear unidad	Prioridad alta			29/03/2024	6
cu-06 Mover unidad	Prioridad alta			26/03/2024	11
cu-09 Finalizar turno	Prioridad baja			11/03/2024	9
cu-21 Atacar	Prioridad alta		10%	01/04/2024	6
Iteración 2	Hito			07/04/2024	31
cu-07 Construir	Prioridad alta		60%	08/04/2024	13
cu-08 Reparar	Prioridad media		50%	05/04/2024	9
cu-10 Listar unidades y construcciones	Prioridad media		33%	10/04/2024	11
cu-11 Describir elemento	Prioridad media			11/04/2024	4
cu-16 Extraer recursos	Prioridad media			12/04/2024	24
cu-17 Almacenar recursos	Prioridad media			13/04/2024	10
cu-18 Agrupar unidades	Prioridad baja			14/04/2024	3
cu-19 Desagrupar unidades	Prioridad baja			15/04/2024	7
Iteración 3	Hito			08/05/2024	31
cu-12 Proponer alianza	Prioridad baja			09/05/2024	4
cu-13 Aceptar alianza	Prioridad baja			10/05/2024	14
cu-14 Revocar alianza	Prioridad baja			11/05/2024	6
cu-15 Rechazar propuesta de alianza	Prioridad media			12/05/2024	3
FIN				08/06/2024	

Los desarrollos se han dividido en tres etapas. En la primera se han desarrollado los caso de uso de importancia “alta”, mientras que las siguientes iteraciones han desarrollado los casos de prioridad media y baja respectivamente.

2. Fase de Elaboración

Esta segunda fase del diseño ocupó las cuatro primeras semanas. En ella se desarrolló un modelo de Casos de Uso que expresa qué funcionalidad va a tener el sistema del videojuego, y un modelo de Vocabulario que recoge la

estructura del dominio de aplicación, mostrando los conceptos importantes y cómo estos se relacionan entre sí.

a. Modelado de Casos de Uso

A continuación se definen y describen los casos de uso de los que consta el diagrama de Casos de Uso.

Código	cu-01
Nombre	Elegir tipo de tablero
Actor principal	Administrador
Precondiciones	Hay suficientes jugadores para empezar la partida (2 o más)
Postcondiciones	- Se ha elegido un tablero. - Se muestra el tipo de tablero seleccionado.
Escenario principal	Se elige una de las tres opciones disponibles.
Escenarios alternativos	-
Prioridad	Alta
Descripción	Antes de iniciar la partida el Administrador deberá decidir si se hace uso del tablero por defecto, si se genera uno nuevo de forma pseudoaleatoria o si se carga un tablero a partir de información previamente almacenada.

Código	cu-02
Nombre	Crear jugador
Actor principal	Usuario
Precondiciones	No se ha sobrepasado el número máximo de jugadores (4). La partida no ha comenzado.
Postcondiciones	Se muestra la lista de jugadores.
Escenario principal	1. Se elige la potencia.

	2. Se registra el jugador.
Escenarios alternativos	- La potencia ya ha sido elegida. Se solicita que el jugador elija otra potencia.
Prioridad	Alta
Descripción	Antes de iniciar la partida cada Usuario debe crear su propio jugador. Puede elegir potencia entre Rusia, China, Europa y Estados Unidos.

Código	cu-03
Nombre	Representar tablero
Actor principal	Usuario
Precondiciones	-
Postcondiciones	-
Escenario principal	Se muestra el tablero por pantalla.
Escenarios alternativos	-
Prioridad	Alta
Descripción	Mostrar el tablero al jugador.

Código	cu-04
Nombre	Iniciar partida
Actor principal	Administrador
Precondiciones	Se han creado dos o más jugadores.
Postcondiciones	-
Escenario principal	El administrador inicia la partida.
Escenarios alternativos	-
Prioridad	Alta
Descripción	El administrador decidirá que se inicia la partida.

Código	cu-05
Nombre	Crear unidad
Actor principal	Usuario
Precondiciones	-
Postcondiciones	Se crea la unidad.
Escenario principal	1. El jugador elige una celda en la que haya una construcción de su propiedad desde la que crear la unidad. 2. El jugador elige un tipo de unidad. 3. El jugador elige una celda en la que colocar la unidad inicialmente.
Escenarios alternativos	1.b El jugador elige una celda no válida. Se solicita que vuelva a elegir celda o cancele la operación de crear unidad. 2.b. El jugador no dispone de los recursos necesarios para crear la unidad. Se solicita que vuelva a elegir tipo de unidad o cancele la operación de crear unidad. 3.b El jugador elige una celda que no es válida para colocar la nueva unidad. Se solicita que vuelva a seleccionar celda o cancele la operación de crear unidad.
Prioridad	Alta
Descripción	El jugador escogerá una construcción que creará una unidad que se posicionará en una de las casillas disponibles

Código	cu-06
Nombre	Mover unidad
Actor principal	Usuario
Precondiciones	Se cumplen las restricciones de movimiento establecidas por el tablero
Postcondiciones	La unidad en cuestión se habrá desplazado en la dirección seleccionada.
Escenario principal	1. Se escoge una unidad 2. Se elige una celda a la que moverse

	3. Se mueve la unidad a la celda
Escenarios alternativos	2.b El jugador elige una celda no válida. Se solicita que vuelva a elegir una celda correcta.
Prioridad	Alta
Descripción	El jugador decide la dirección en la que se moverá la unidad seleccionada

Código	cu-07
Nombre	Construir
Actor principal	Usuario
Precondiciones	-
Postcondiciones	La celda se convierte en no transitable.
Escenario principal	1. Se escoge un robot del jugador. 2. Se indica una dirección de construcción. 3. Se indica el tipo de construcción deseado. 4. Se comprueba si se dispone de los recursos suficientes.
Escenarios alternativos	4.b El jugador no dispone de recursos suficientes para llevar a cabo la construcción. Se cancela la operación. 2.b La posición elegida no admite que el jugador actual cree una construcción en ella. Se solicita que el jugador elija otra casilla o cancele la operación.
Prioridad	Media
Descripción	El jugador decide construir una de las posibles estructuras (estaciones, bases y torres) en una casilla adyacente.

Código	cu-08
Nombre	Reparar
Actor principal	Usuario

Precondiciones	-
Postcondiciones	La construcción reparada aumentará su salud.
Escenario principal	1. Se escoge un robot del jugador.. 2. Se indica la dirección de la construcción a reparar. 3. Se diagnostica el edificio que se encuentra en la celda. 4. Se comprueba la disponibilidad de los recursos. 5. Se repara la construcción haciendo que recupere los puntos de salud.
Escenarios alternativos	2.c La construcción seleccionada no pertenece al jugador actual, por lo que se cancela la operación. 3.b La salud de la construcción elegida ya está completa, por lo que se cancela la operación. 4.d El jugador no dispone de los recursos suficientes para llevar a cabo la reparación, por lo que se cancela la operación.
Prioridad	Media
Descripción	El jugador decide reparar una estructura en una casilla adyacente.

Código	cu-09
Nombre	Finalizar turno
Actor principal	Usuario
Precondiciones	Estar en el turno del usuario.
Postcondiciones	El turno del usuario habrá terminado, dando lugar al del siguiente usuario.
Escenario principal	El turno finaliza.
Escenarios alternativos	1. El jugador no ha llevado a cabo ninguna acción, por lo que se pide una confirmación para terminar el turno.
Prioridad	Alta
Descripción	El jugador acaba su turno.

Código	cu-10
Nombre	Listar unidades y construcciones
Actor principal	Usuario
Precondiciones	-
Postcondiciones	-
Escenario principal	Se listan las unidades y construcciones existentes y visibles para el usuario que la ejecuta.
Escenarios alternativos	-
Prioridad	Media
Descripción	La interfaz lista las unidades y construcciones del juego mostrando su posición.

Código	cu-11
Nombre	Describir elemento
Actor principal	Usuario
Precondiciones	El elemento seleccionado es visible para la potencia que dispone del turno.
Postcondiciones	-
Escenario principal	1. Se elige la celda a describir o en la que está el elemento a describir. 2. Se describe la celda y el elemento que pueda haber en ella.
Escenarios alternativos	1.b La celda seleccionada no es visible para el jugador. Se solicita que se vuelva a elegir celda o se cancele la operación de describir elementos.
Prioridad	Media
Descripción	Se describe la celda seleccionada por el usuario y la unidad o unidades que pueda haber en ella siempre y cuando sea visible para el mismo.

Código	cu-12
Nombre	Proponer alianza
Actor principal	Usuario
Precondiciones	-
Postcondiciones	Se guarda la propuesta de alianza del jugador a otro.
Escenario principal	1. Se escoge un jugador. 2. Se le propone una alianza.
Escenarios alternativos	-
Prioridad	Baja
Descripción	El jugador puede proponer alianza a otro.

Código	cu-13
Nombre	Aceptar alianza
Actor principal	Usuario
Precondiciones	Tener una propuesta de alianza y estar al inicio del turno.
Postcondiciones	Se fija la alianza entre los dos jugadores.
Escenario principal	1. Se escoge una alianza. 2. Se acepta la alianza.
Escenarios alternativos	2.b De tener el jugador ya una alianza, se revoca y se acepta la nueva.
Prioridad	Baja
Descripción	El jugador acepta la propuesta de alianza de otro jugador.

Código	cu-14
Nombre	Revocar alianza
Actor principal	Usuario
Precondiciones	Tener una alianza con otro jugador.
Postcondiciones	Se elimina la alianza entre los dos jugadores.

Escenario principal	1. Se elimina la alianza existente entre los dos jugadores.
Escenarios alternativos	-
Prioridad	Baja
Descripción	El jugador revoca la alianza.

Código	cu-15
Nombre	Rechazar propuesta de alianza
Actor principal	Usuario
Precondiciones	Existe alguna propuesta de alianza dirigida a él y es el inicio del turno del jugador.
Postcondiciones	Queda eliminada la propuesta de alianza.
Escenario principal	1. El jugador elige una de las alianzas que se le han propuesto. 2. Se elimina la propuesta de alianza.
Escenarios alternativos	-
Prioridad	Baja
Descripción	El jugador puede, al inicio de su turno, rechazar una propuesta de alianza que se le haya hecho.

Código	cu-16
Nombre	Extraer recursos
Actor principal	Usuario
Precondiciones	Disponer de algún robot en disposición de extraer recursos
Postcondiciones	Los recursos serán restados a la casilla y sumados al robot
Escenario principal	1. Se elige un robot del jugador. 2. Se elige una celda. 3. Se extraen los recursos de la celda.
Escenarios alternativos	1.b El robot escogido ya ha alcanzado el

	máximo de recursos que puede transportar. Se solicita que lleve al robot a una estación. 2.b La celda escogida no es válida o no tiene recursos. Se solicita que vuelva a elegir otra celda.
Prioridad	Media
Descripción	El jugador elige una unidad para que extraiga recursos de una casilla.

Código	cu-17
Nombre	Almacenar recursos
Actor principal	Usuario
Precondiciones	Hay recursos previamente extraídos disponibles para almacenar.
Postcondiciones	-
Escenario principal	1. Se elige un robot del jugador 2. Se elige una estación del jugador. 3. Se almacenan los recursos en la estación.
Escenarios alternativos	1.b El robot escogido no tiene recursos que almacenar. Se avisa que el robot no tiene recursos. 2.b La estación escogida no es válida. Se solicita que elija otra.
Prioridad	Media
Descripción	El jugador elige una unidad para que almacene sus recursos en una estación.

Código	cu-18
Nombre	Agrupar unidades
Actor principal	Usuario
Precondiciones	-
Postcondiciones	Las unidades quedan agrupadas y no se pueden seleccionar individualmente, solo se puede seleccionar todo el grupo.
Escenario principal	1. Se elige una casilla. 2. Se agrupan todas las unidades de esa

	casilla.
Escenarios alternativos	2.b Solo hay una o ninguna unidad en esa casilla. No se pueden agrupar.
Prioridad	Baja
Descripción	El jugador selecciona las unidades que formarán un grupo cuya capacidad de ataque y defensa vendrá dada por la suma de capacidades de sus integrantes

Código	cu-19
Nombre	Desagrupar unidades
Actor principal	Usuario
Precondiciones	Tener un grupo de unidades.
Postcondiciones	El grupo de unidades se desagrupa y cada unidad puede ser seleccionada individualmente.
Escenario principal	1. Se elige un grupo de unidades. 2. Se desagrupa.
Escenarios alternativos	1.b La unidad seleccionada no es un grupo de unidades. Tienes que volver a seleccionar.
Prioridad	Baja
Descripción	El jugador

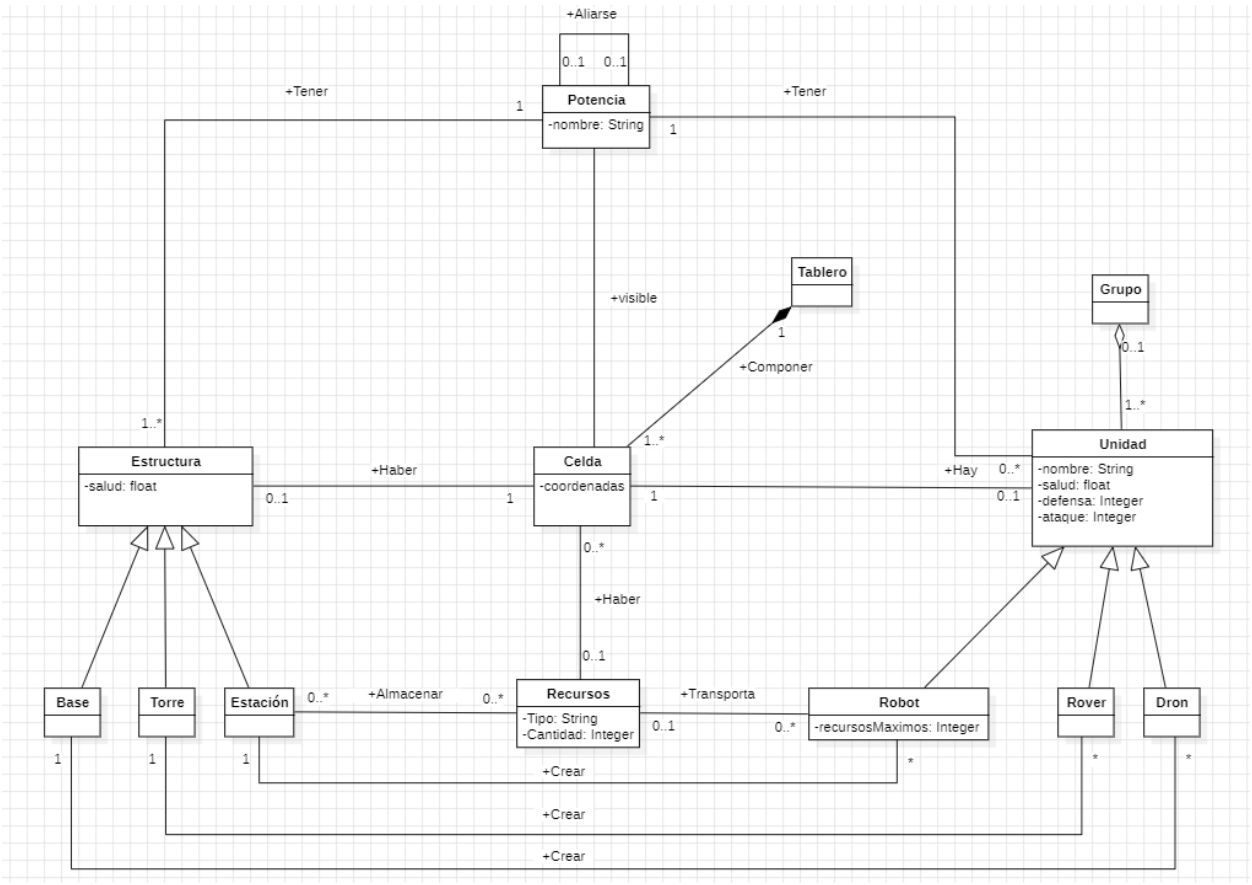
Código	cu-20
Nombre	Listar potencias
Actor principal	Usuario
Precondiciones	-
Postcondiciones	-
Escenario principal	1. Se listan todas las potencias.
Escenarios alternativos	-
Prioridad	Baja
Descripción	Listar las potencias que están actualmente

	en juego (que dispongan de 1 o más estaciones)

Código	cu-21
Nombre	Atacar
Actor principal	Usuario
Precondiciones	-
Postcondiciones	Se reduce la vida de la unidad atacada.
Escenario principal	1. Se elige una unidad del jugador. 2. Se indica una dirección a la que atacar. 3. Se ataca.
Escenarios alternativos	1.b. La unidad seleccionada no pertenece al jugador. Se pide que se seleccione otra unidad. 2.b. Se selecciona una celda en la que no hay una unidad enemiga (no hay ninguna unidad o la unidad que hay es aliada o propia). Se pide que se seleccione otra celda. 3.b El enemigo neutraliza el ataque. Se avisa de que el ataque no ha tenido efecto. 3.c Se derrota la última unidad de una potencia. Se elimina la potencia. 3.d Se derrota la última unidad de la última potencia enemiga. Se elimina la potencia y el jugador gana la partida.
Prioridad	Media
Descripción	El jugador selecciona una unidad y una celda a la que está atacará.

b. Modelo de Vocabulario

Se muestra una captura de este modelo, que captura los conceptos de mayor relevancia y las relaciones que se establecen entre ellos.



3. Fase de Construcción

En esta fase, cuya duración fue de dos meses, es en la que se inicia el desarrollo del diseño en sí, creando varios diagramas de clases y de secuencia.

a. Planificación

Como punto de inicio para la planificación y la organización de esta tercera fase se usa el Modelo de Casos de Uso creado en la primera fase del proyecto, partiendo de cada uno para desarrollar los diferentes diagramas de secuencia que muestran el procedimiento de ejecución del videojuego. Esta fase se divide en tres iteraciones.

b. Iteración 1

Esta primera iteración, con fecha de inicio el 6 de marzo y de un mes de duración, tiene como objetivo el desarrollo de los Casos de Uso

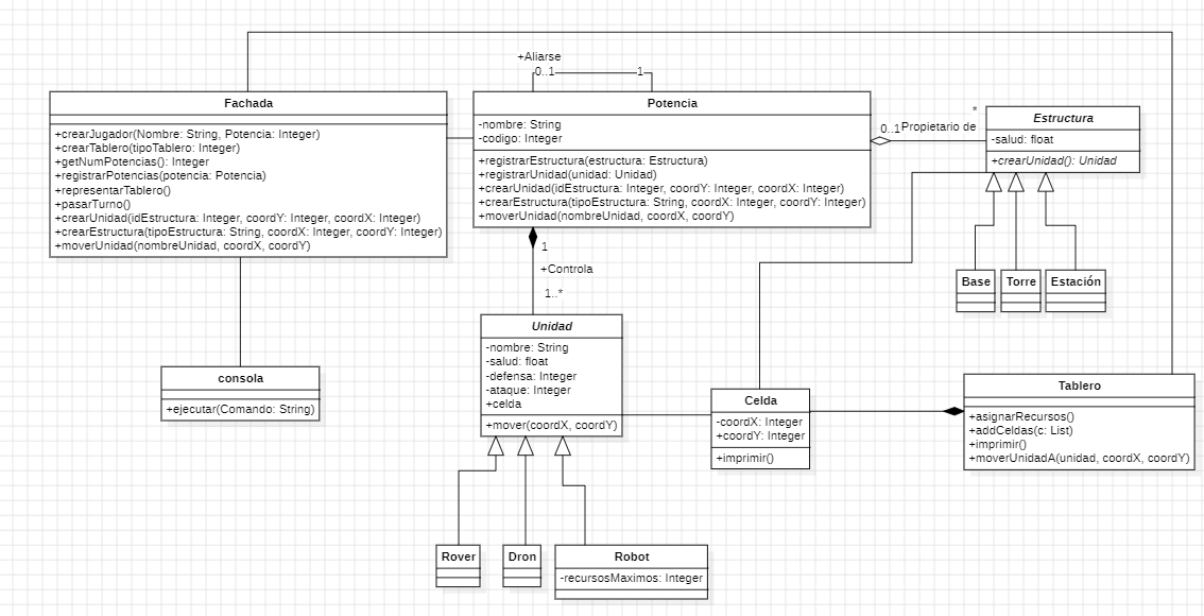
más básicos y necesarios para la iniciación del juego, aquellos de prioridad alta:

Casos de uso

cu-01 Elegir tipo de tablero
cu-02 Crear jugador
cu-03 Representar tablero
cu-05 Crear unidad
cu-06 Mover unidad
cu-09 Finalizar turno
cu-21 Atacar

Estructura

En primer lugar se creó un diagrama estructural para mostrar el diseño:

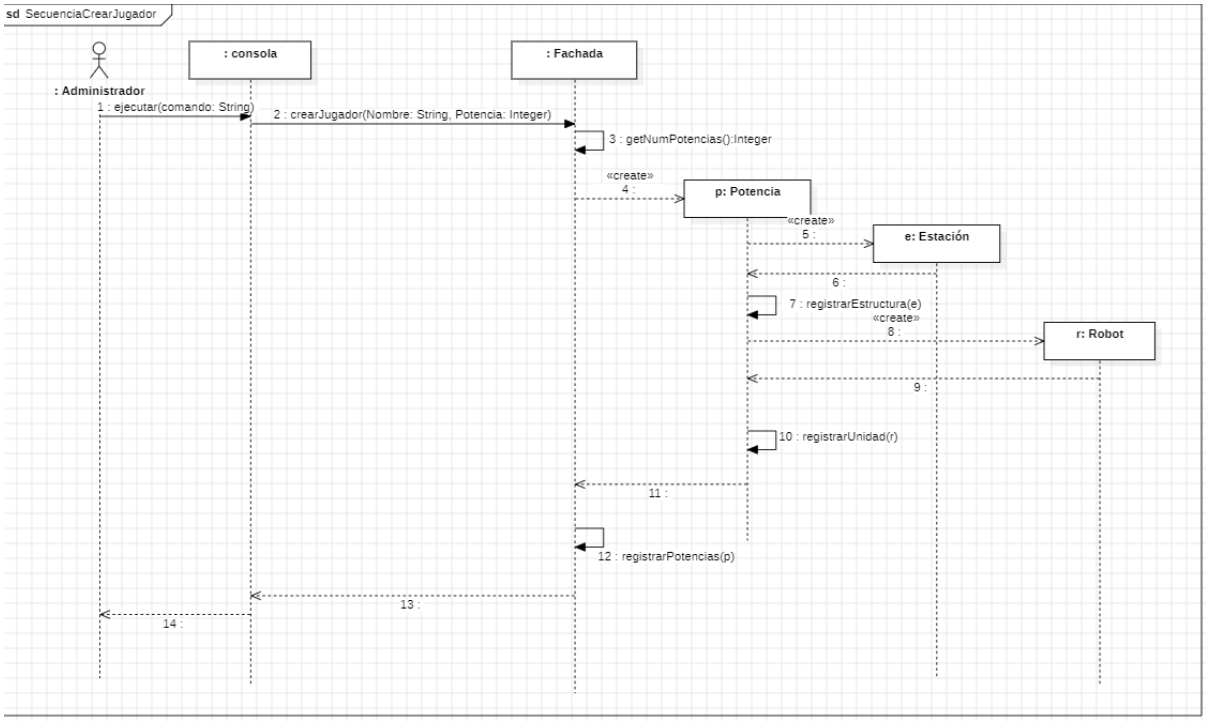


En este se muestra una proposición de los atributos y funcionalidades que tendrían cada una de las clases necesarias para la primera iteración.

Se siguió el patrón de diseño *Facade*, por el que se define una clase superior “Fachada”, que encapsula toda la funcionalidad compleja del programa, facilitando la modularidad y la encapsulación de la misma.

Comportamiento

El comportamiento deseado se presenta en los diagramas de secuencia, habiendo creado uno por cada Caso de Uso de esta Iteración. Como ejemplo, se muestra el diagrama de secuencia creado para “Crear jugador”:



En este diagrama de secuencia se muestra como el Administrador, encargado de crear a los jugadores, escribe la sentencia de creación de nuevo jugador a consola, que a su vez ejecuta el comando “Crear jugador” con los parámetros necesarios para crear un nuevo jugador. Antes de continuar el proceso se comprueba con “getNumPotencias()” en la Fachada que no se sobrepasa el máximo de Potencias para el juego. Tras esto desde la fachada se crea una nueva instancia de la clase Potencia que se registra en la Fachada y que, a su vez, crea una nueva instancia de Estación desde donde el jugador podrá crear sus tropas. Esta estación se registra en la potencia. Por último se crea un Robot inicial que también se registra en la potencia.

Este patrón de diseño se implementó también con el patrón de creación *Singleton*, por el que, en el programa, únicamente se creará una instancia de Fachada, y tendrá la operación estática, `getInstance()`, como punto de acceso global a ella.

// Ejemplo?

Se implementó otro patrón de creación más, *Abstract Factory*, por el que la clase Estructura define una operación abstracta `crearUnidad()`, que será concretada en las especificaciones Torre, Base y Estación, instanciando un objeto Rover, Dron o Robot, respectivamente.

// Ejemplo?
En cuanto a patrones de estructura, además de *Facade*, se implementó el patrón *Composite*, en tanto que Unidad es una clase abstracta cuyas especificaciones Rover, Dron y Robot (partes) pueden ser agrupadas en un GrupoUnidades (todo), clase definida como una agregación de instancias Unidad, pero una instancia GrupoUnidades también puede formar, a su vez, parte de otro GrupoUnidades.

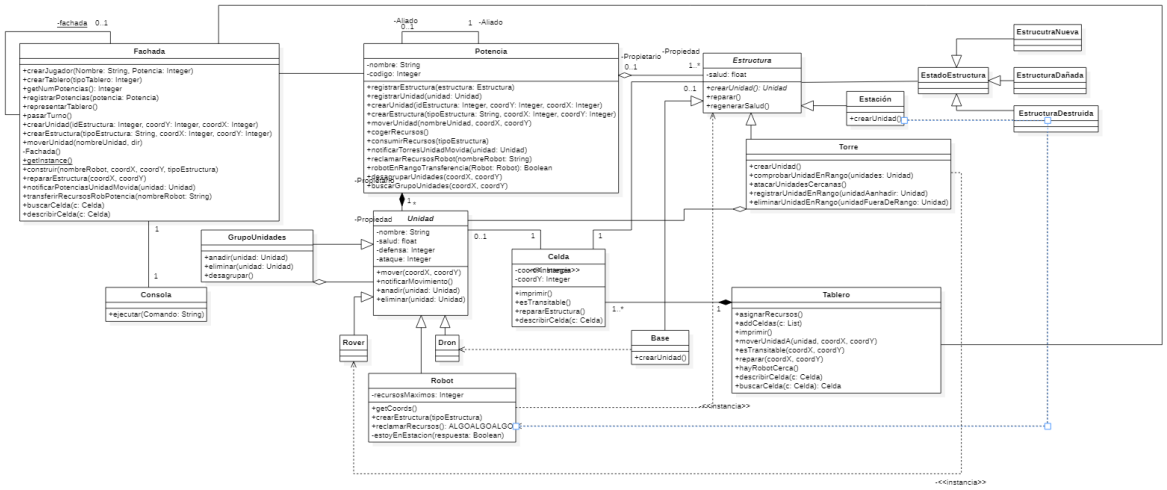
c. Iteración 2

Para esta segunda iteración, con fecha de inicio el 8 de abril y duración de cerca de un mes, el foco se pondrá en los casos de uso de prioridad media.

Casos de uso

cu-07 Construir
cu-08 Reparar
cu-10 Listar unidades y construcciones
cu-11 Describir elemento
cu-16 Extraer recursos
cu-17 Almacenar recursos
cu-18 Agrupar unidades
cu-19 Desagrupar unidades

Estructura



La diferenciación de estados que puede tener una instancia de Estructura se resolvió por medio del patrón de comportamiento *State*. Así, cada instancia de Estructura tendrá asociado un EstadoEstructura, que cambiará según el estado de la estructura: EstructuraNueva, EstructuraDañada o EstructuraDestruida, según si la salud de la Estructura está al máximo, si no está al máximo pero aún no está destruida, y destruida, respectivamente.

Para la funcionalidad de ataque automático de las torres se siguió el patrón Observer, definiendo una relación sujeto observado - observadores, en la que el sujeto observado sería cada Unidad cuando se desplaza por el mapa, y los observadores serían cada Torre de una potencia diferente a la propietaria de la Unidad. Cuando una Unidad se desplace por el mapa ejecutará la opción `notificarMovimiento()`, para notificar a las Torres enemigas de que ha variado la posición de esa Unidad. Cada Torre evaluará, ejecutando `comprobarUnidadEnRango()`, si la Unidad que ha notificado movimiento está en rango de ataque, incluyéndola en la lista de unidades en rango con `registrarUnidadEnRango()`, o si ya no está, eliminándola de la lista de unidades en rango con `eliminarUnidadEnRango()`. Si no se da ninguno de estos casos no hará nada.

// EJEMPLO

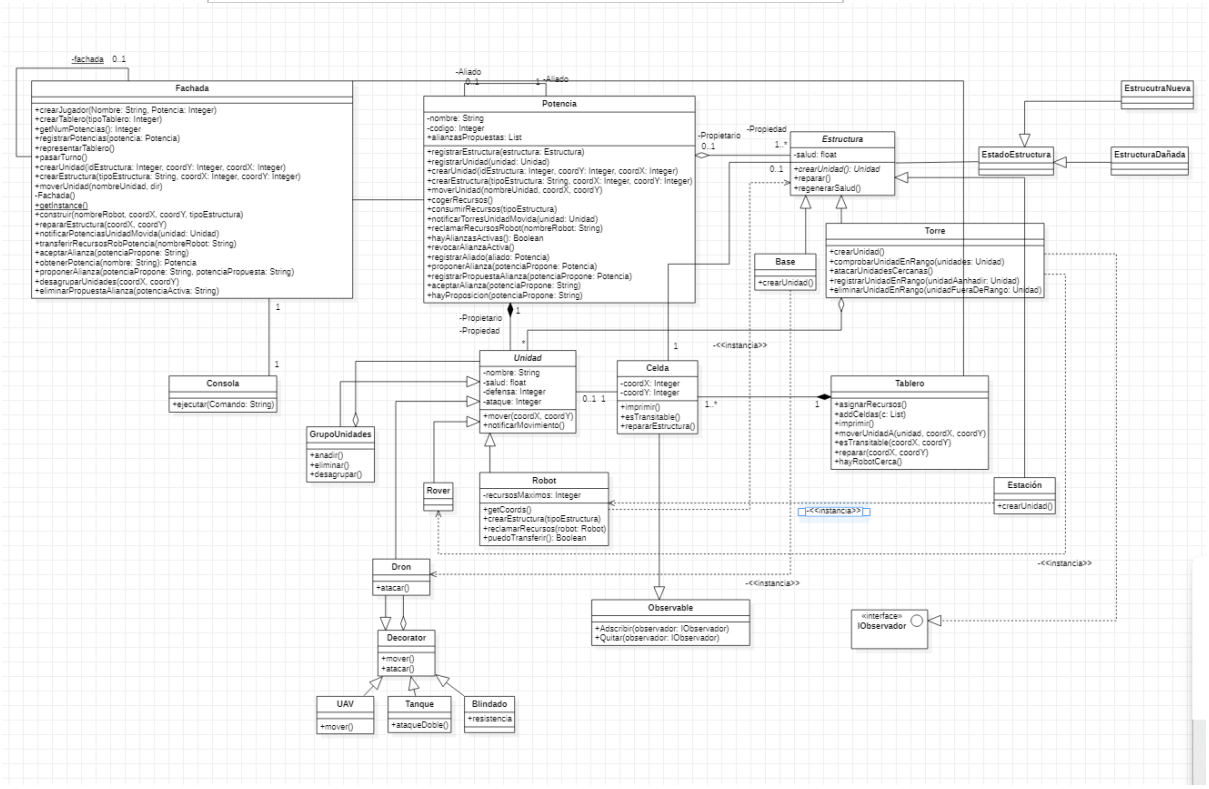
delegacion

// decorator -> tipos de drones

d. Iteración 3

En esta última iteración el objetivo es el de llevar a cabo del desarrollo de la funcionalidad restante, de prioridad baja:

cu-12 Proponer alianza
cu-13 Aceptar alianza
cu-14 Revocar alianza
cu-15 Rechazar propuesta de alianza



4. Fase de Transición

APÉNDICE 1: Tarjetas CRC

Nombre de la clase: **Consola**	
Finalidad: **Interacción con el usuario**	
Responsabilidades:	**Colaboradores**:
Recibir comandos del exterior	Fachada
Pasar las ordenes al sistema	Fachada

Nombre de la clase: **Potencia**	
Finalidad: **Representa el rol de un presidente de una Potencia mundial**	
Responsabilidades:	**Colaboradores**:
Construir estructuras	Unidad (robot)
Realizar alianzas	-
Extraer recursos	Unidad (robot)
Ensamblar autómatas programables	Estructura

Nombre de la clase: ***Unidad***	
Finalidad: **Modelo genérico para un autómata programable**	
Responsabilidades:	**Colaboradores**:
Moverse	Celda
Agruparse	GrupoUnidades

Nombre de la clase: **GrupoUnidades**	
Finalidad: **Permitir la creación de grupos de unidades**	
Responsabilidades: Realizar una acción (existente en unidad)	**Colaboradores**: Unidad

Nombre de la clase: **Dron**	
Finalidad: **Realizar ataques a otras unidades/edificaciones**	
Responsabilidades: Moverse Realizar ataques	**Colaboradores**: Celda Tablero, Estructura | Unidad

Nombre de la clase: **DronDecorator**	
Finalidad: **Permite expandir la funcionalidad de los drones mediante su mejora**	
Responsabilidades: Ampliar las capacidades de los drones	**Colaboradores**: UAV, Tanque, Blindado

Nombre de la clase: **UAV**	
Finalidad: **Extensión de la funcionalidad de los drones, mejorando la habilidad de movimiento**	
Responsabilidades:	**Colaboradores**:

Nombre de la clase: **Tanque**	
Finalidad: **Extensión de la funcionalidad de los drones, mejorando la habilidad para atacar**	
Responsabilidades:	**Colaboradores**:

Nombre de la clase: **Blindado**	
Finalidad: **Extensión de la funcionalidad de los drones, mejorando la resistencia**	
Responsabilidades:	**Colaboradores**:

Nombre de la clase: **Rover**	
Finalidad: **Especialización de Unidad para la exploración del terreno**	
Responsabilidades:	**Colaboradores**:

Nombre de la clase: **Robot**	
Finalidad: **Unidad especializada para la recolección de recursos y construcción de estructuras**	
Responsabilidades:	**Colaboradores**:
Extraer recursos	Celda
Transferir recursos	Celda, Potencia
Construir estructuras	Potencia, Estructura

Nombre de la clase: **Estructura**		
Finalidad: **Modelo genérico para una edificación**		
Responsabilidades:		**Colaboradores**:
Crear unidades		-
Reparar edificaciones		-

Nombre de la clase: **Base**		
Finalidad: **Especialización de edificación**		
Responsabilidades:		**Colaboradores**:
Crear unidades		-
Reparar edificaciones		-

Nombre de la clase: **Torre**		
Finalidad: **Especialización defensiva de edificación**		
Responsabilidades:		**Colaboradores**:
Atacar unidades		Unidad
Atacar unidades en automático		

Nombre de la clase: **Estación**
Finalidad: **Especialización de edificación**

Responsabilidades:	**Colaboradores**:
Atacar unidades Atacar unidades en automático	Unidad

Nombre de la clase: **Celda**	
Finalidad: **Aporta las localización de entidades en el mapa**	
Responsabilidades:	**Colaboradores**:
Albergar Unidades	Unidad
Albergar Estructuras	Estructura
Albergar Recursos	-

Nombre de la clase: **Observable**	
Finalidad: **Clase genérica para permitir observadores**	
Responsabilidades:	**Colaboradores**:

Nombre de la clase: **Tablero**	
Finalidad: **Gestionar las celdas y lo que contienen**	
Responsabilidades:	**Colaboradores**:

Nombre de la clase: **Torre**	
Finalidad: **Especialización defensiva de edificación**	
Responsabilidades:	**Colaboradores**:
Atacar unidades	Unidad
Atacar unidades en automático	Unidad, IObservador

Nombre de la clase: **EstadoEstructura**	
Finalidad**: Implementación del patrón "*state*" para las estructuras**	
Responsabilidades:	**Colaboradores**:

Nombre de la clase: **EstructuraNueva**	
Finalidad**: Estado específico de EstadoEstructura (clase hija)**	
Responsabilidades:	**Colaboradores**:

Nombre de la clase: **EstructuraDañada**	
Finalidad**: Estado específico de EstadoEstructura (clase hija)**	
Responsabilidades:	**Colaboradores**: